



第三章 软件工程技术

- 3.1 软件开发与软件工程
- 3.2 软件危机与软件工程框架及原则
- 3.3 软件生命周期及软件开发模型
- 3.4 软件开发流程
- 3.5 模块化设计技术
- 3.6 图形表示技术





3.1 软件开发与软件工程

3.1.1 软件开发的演化过程

1) 个人编程时代(46年~50年代末)

- 软件开发是科学家们根据各自的应用需要写出的能够解决预定问题的运行程序。程序生产的效率极低，可靠性难以保证，且仅限于处理比较简单的数值计算问题。

2) 软件作坊时代(60年代初~60年代末)

- 软件作坊的开发方法是个体的或小组的思维行为，使得软件任务延误、质量不可靠、甚至无法维护，极大地制约了计算机以后的功能发挥和实际应用。

3) 软件工程时代(70年代~80年代)

- 在世界范围内出现了许多组织严密、管理科学、手段先进、工具齐全的软件开发公司，为计算机软件市场提供了大量成功的软件产品。80年代，明确提出了“软件工程支撑环境”的思想，使程序设计可以直接从支撑环境中调用所需的各个“组件”。





1. 软件工程与编程的区别

软件工程

- 是一门学科，一种科学理论，用来指导软件系统的开发，以实现软件开发过程的**标准化，自动化**。
- 考虑如何分解一个系统，以便分工开发；考虑如何说明每个部分的规格要求；怎样才能易于维护。

编程

- 单纯的代码编写
- 是软件工程发展的前身
- 是软件工程中占据很少时间和空间的一部分





3.2 软件危机与软件工程

- 一. 软件的定义
- 二. 软件的特点
- 三. 软件危机
- 四. 软件工程框架及原则
- 五. “软件工程”课程教学的目标





一. 软件的定义

- 软件是计算机程序、方法、规则、相关的文档以及在计算机上运行时所需要数据的总称。软件是计算机系统的核心。

二. 软件的特点

1. 软件是一种逻辑实体，是逻辑产品
2. 没有硬件那样的机械磨损、老化问题。





1. 开发和运行常常受计算机系统的限制

① 硬件限制

I. 286时代不可能开发出UG。

② 软件限制

I. DOS时代不可能开发出具有视窗界面的应用程序。

2. 手工作坊式的开发方式（生产率低）

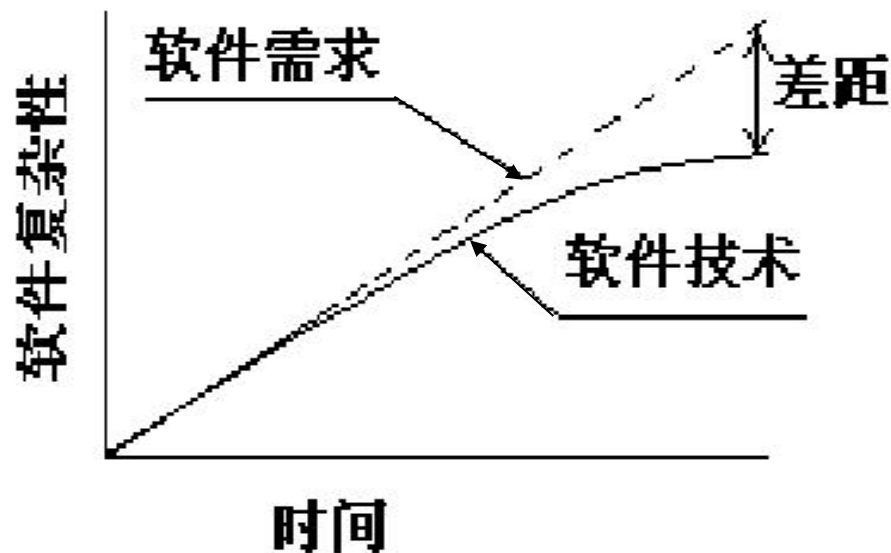
① 虽然模块化、动态连接库技术、组件技术等，都在致力于代码的重用，但都要进行或多或少的手工修改。不能象机械零件（如：螺钉、螺母）那样，拿来就用。





1. 软件技术的发展落后于需求

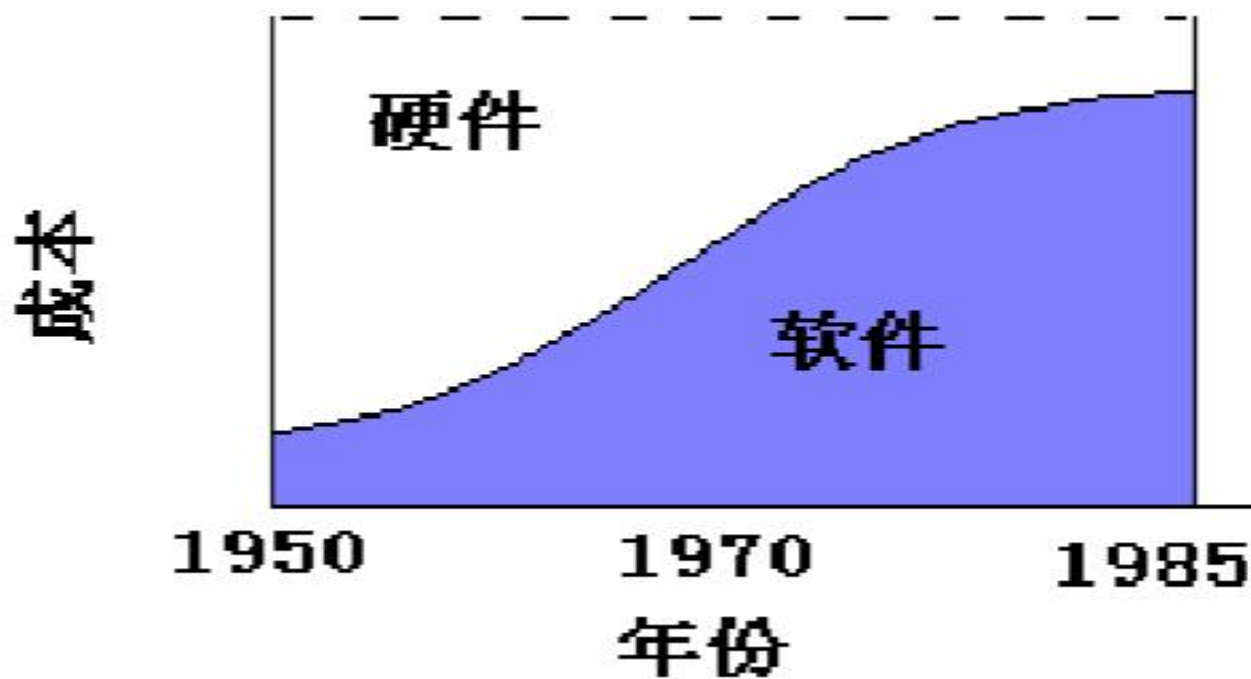
① 软件设计越来越复杂性





1. 软件开发的成本高

计算机系统硬、软件成本比例的变化





1. 软件的规模越来越大

类别	参加人数	研制期限	产品规模（源程序行数）
微型	1	1~4周	0.5k
小型	1	1~6月	1k~~2k
中型	2~5	1~2年	5k~50k
大型	5~20	2~3年	50k~100k
超大型	100~1000	4~5年	1M(1000k)
极大型	2000~5000	5~10年	1M~10M





① 例如：Windows代码量

I. Windows95有1000万行代码

II. Windows2000有5000万行代码

② 例如：Exchange2000和Windows2000开发人员结构

	Exchange2000	Windows2000
项目经理	25人	约250人
开发人员	140人	约1700人
测试人员	350人	约3200人





一. 软件危机

■ 指在计算机软件开发和维护过程中所遇到的一系列问题。

- ① 不能正确地估计软件开发成本和进度，致使实际开发成本往往高出预算很多。
- ② 软件产品不可靠，满足不了用户的需求，甚至无法使用。
- ③ 交付使用的软件不易演化，以至于人们不得不重复开发类似的软件。
- ④ 软件生产率低下，远远满足不了社会发展的需求。



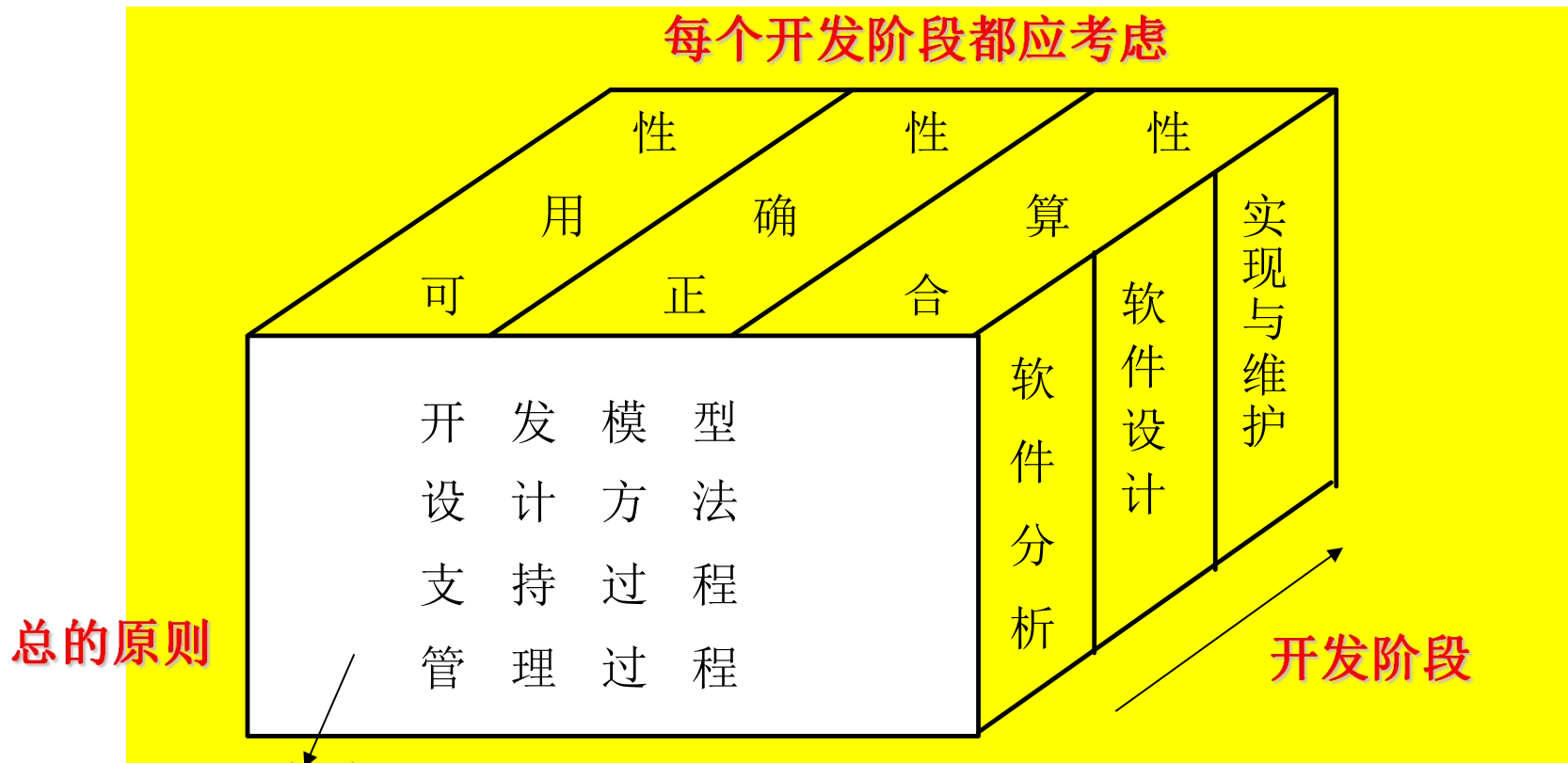


■ 例子：

- ① 美国**IBM**公司在1963年~1966年开发的**IBM 360**机的操作系统。
- ② 该项目花了**5000人/年**的工作量，最多时有**1000**人同时投入开发工作，写出了近**100万行**源程序。
- ③ 尽管投入了这样多的人力物力，得到的结果却非常糟糕。据统计，这个操作系统每次发行新版本都是从前一版本中找出**1000个**程序错误修正的结果。
- ④ 项目负责人**F.D.Brooks**事后总结了他在组织开发过程中的沉痛教训时说：“.....正像一只逃亡的野兽落到泥潭中作垂死的挣扎一样，越是挣扎，陷得越深。最后也无法逃脱灭顶的灾难，.....程序设计工作正像这样一个泥潭，.....一批批程序员被迫在泥潭中拼命挣扎，.....谁也没有料到问题竟会陷入这样的困境.....”。
- ⑤ **IBM 360** 操作系统的历史教训成为软件开发项目的典型事例为人们所记取。



一. 软件工程框架及原则



原则：

- 1、选取适宜的开发模型；
- 2、采用合适设计方法；
- 3、提供高质量工程支持；
- 4、重视开发过程管理。





一. “软件工程”课程教学的目标

1. 转变对软件的认识:

上升

程序 → 系统

2. 转变思维定式:

上升

程序员 → 系统工程师
(系统分析员)

3. 实现

进而

软件开发过程的标准化 → 软件产品的标准化





3.3 软件生命周期及软件开发模型

- 软件生命周期
 - 软件生命周期是指软件产品或软件系统从设计、投入使用到被淘汰的全过程。
- 软件开发模型（软件过程模型）
 - 软件开发模型是指在软件的整个生命期中，软件开发全部过程、活动和任务的结构框架。





3.3.1 软件生命周期

- 软件生命周期中阶段划分
 - 1. 可行性研究与计划 } 孕育期（计划期）
 - 2. 需求分析
 - 3. 总体设计
 - 4. 概要设计
 - 5. 详细设计 } 成长期（开发期）
 - 6. 实现
 - 7. 集成测试
 - 8. 确认测试
 - 9. 使用和维护 } 成年期（运行期）
 - 10. 退役





- 说明

1. 设计过程上采用了现代产品设计的理念。
2. 软件开发的范围

只考虑

编写程序

扩展到

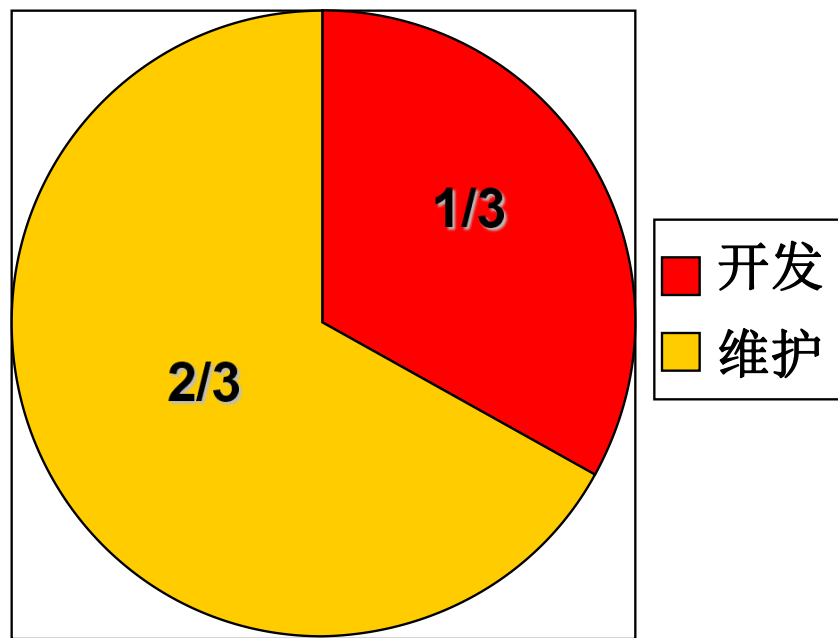


涉及整个软件生存周期的全过程

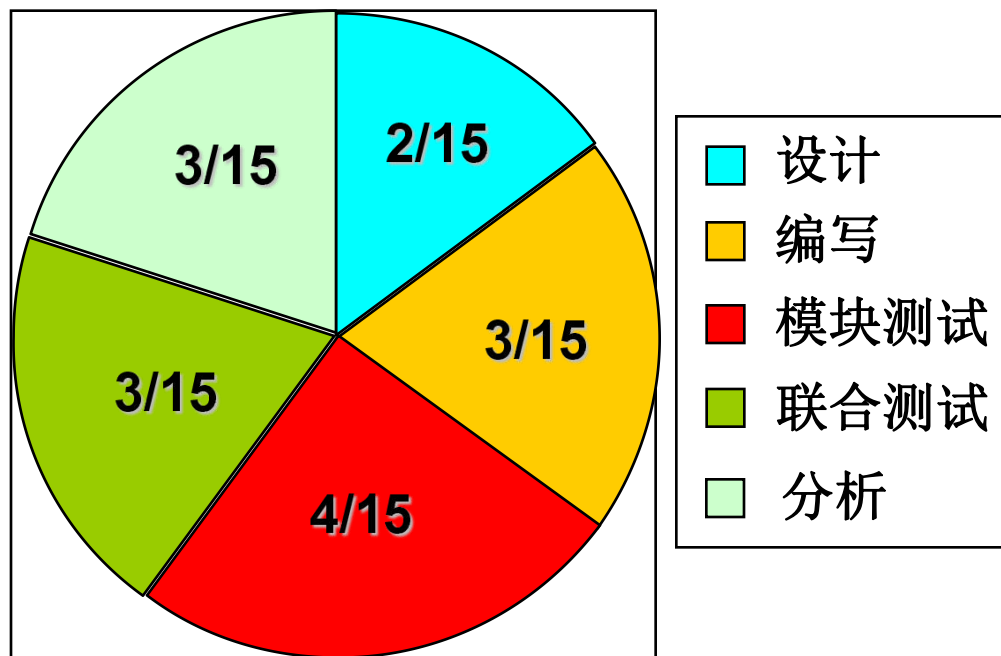




• 软件生命周期中各阶段所占的比重



按生命周期划分

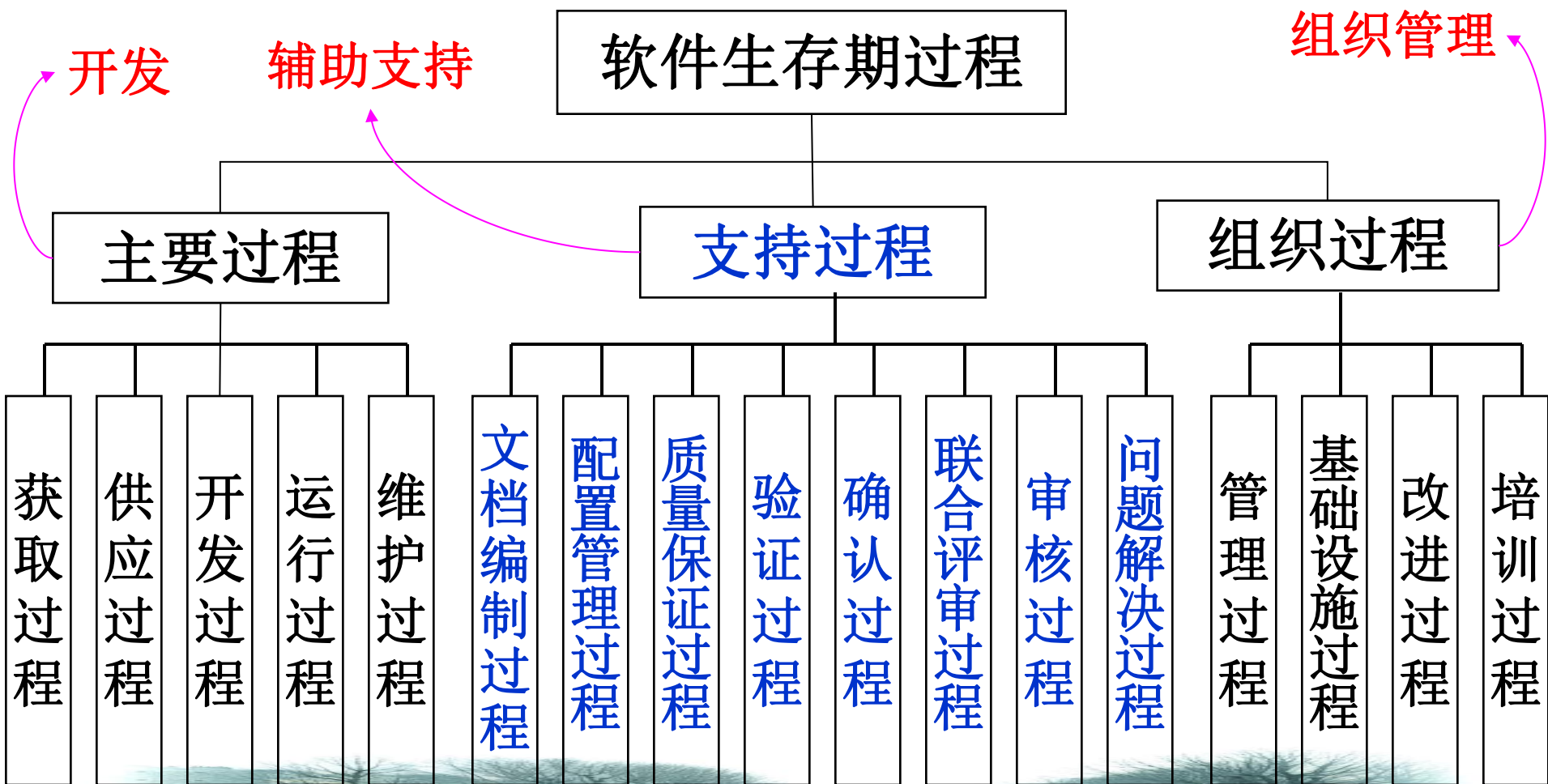


按开发阶段划分





新的国际标准定义的软件生存过程 (1995 ISO 12207)





3.3.2 软件开发模型

一. 软件开发模型

1. 软件开发模型是软件开发全部过程、活动和任务的**结构框架**。
2. 它能直观表达软件开发全过程，明确规定要完成的主要活动、任务和开发策略。
3. 软件开发模型也常称为：**软件过程模型**、**软件生存周期模型**、**软件工程范型**。

二. 常用的软件开发模型

- **瀑布模型**（线性顺序模型）、**原型模型**、**增量模型**、**螺旋模型**



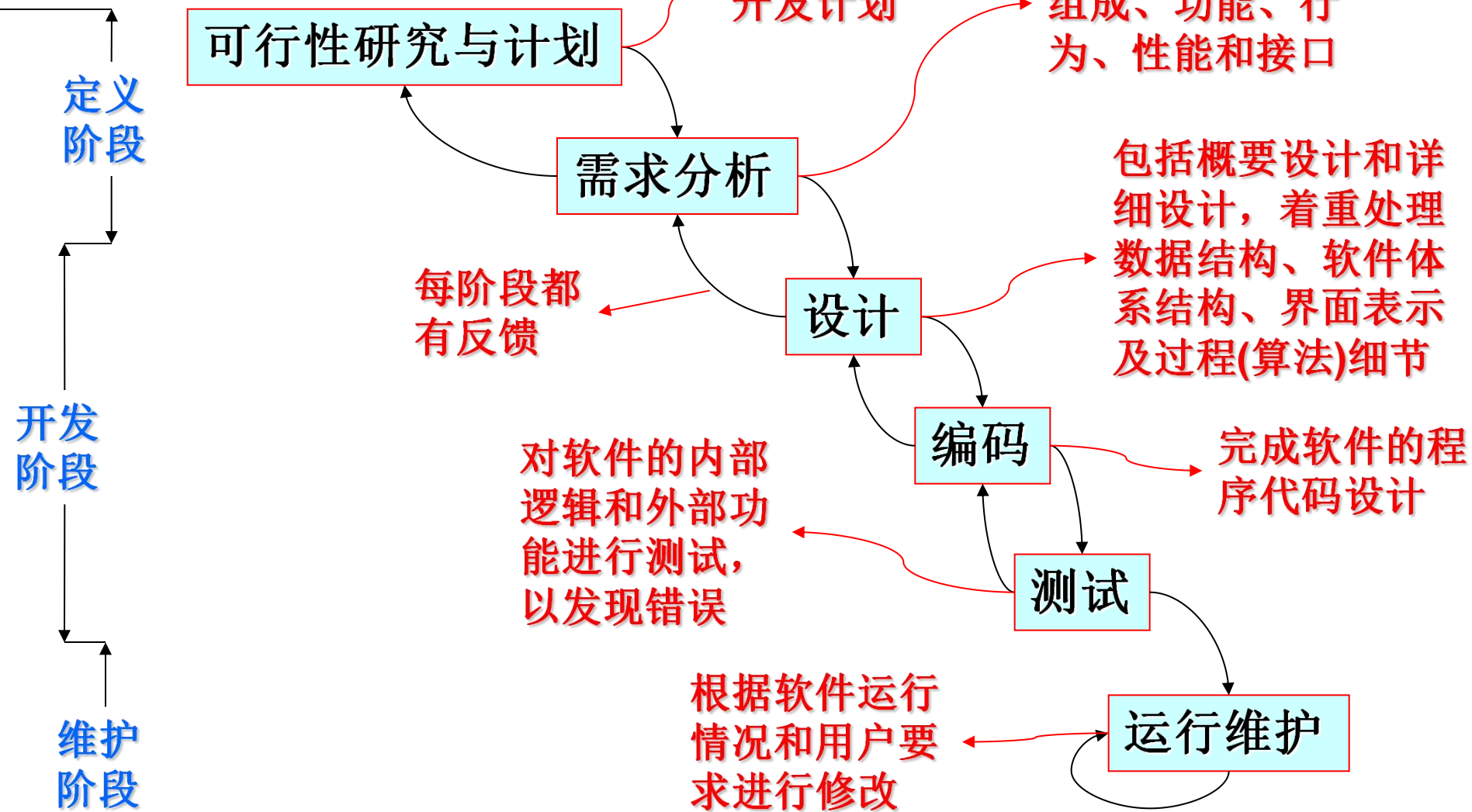


一. 软件开发模型简介

1. 瀑布模型(Waterfall Model)

- 由W.Royce于1970年首先提出。
- 根据软件工程生存周期各个阶段的任务，瀑布模型从可行性研究开始，逐步进行阶段性变换，直至通过确认测试并得到用户确认的软件产品为止。
- 瀑布模型上一阶段的变换结果是下一阶段变换的输入，相邻两个阶段具有因果关系，紧密联系。一个阶段的失误将蔓延到以后的各个阶段。
- 为了保障软件开发的正确性，每一阶段任务完成后，都必须对它的阶段性产品进行评审，确认之后再转入下一阶段的工作。
- 评审过程发现错误和疏漏后，应该及时反馈到前面的有关阶段修正错误或弥补疏漏，然后再重复前面的工作，直至某一阶段通过评审后再进入下一阶段。
- 顺序执行







① 瀑布模型的特点

I. 优点：

- 可强迫开发人员采用规范的方法；
- 严格规定了每个阶段必须提交的文档；
- 要求每个阶段交出的所有产品都必须经过质量保证小组的仔细验证等。

I. 缺点：

- 在软件开发的初始阶段指明软件系统的全部需求是困难的，有时甚至是不现实的。而瀑布模型在需求分析阶段要求客户和系统分析员必须做到这一点才能开展后续阶段的工作。
- 确定需求后，用户和软件项目负责人要等相当长的时间才能得到一份软件的最初版本。如果用户对这个软件提出比较大的修改意见，那么整个软件项目将会蒙受巨大的人力、财力和时间方面的损失。

② 适用范围

- ### I. 适合于功能和性能明确且无重大变化的软件开发。





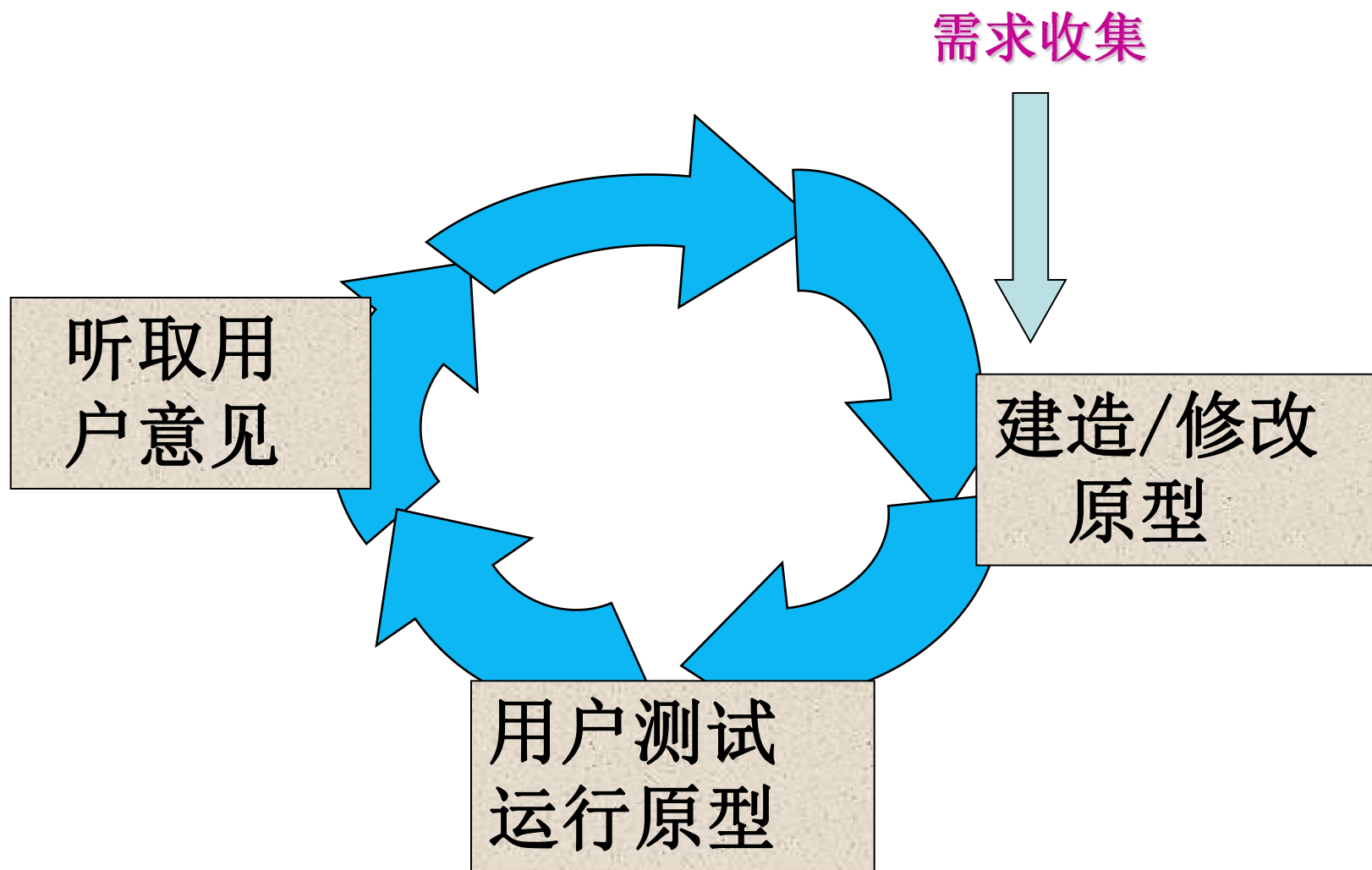
1. 原型模型

- 原型模型又称演化模型，主要是针对事先不能完整定义需求的软件项目开发而言的。
- 许多软件开发项目由于人们对软件需求的认识模糊，很难一次开发成功，返工再开发难以避免。
- 因此，人们对需开发的软件给出基本需求，作第一次试验开发，其目标仅在于探索可行性和弄清需求，取得有效的反馈信息，以支持软件的最终设计和实现。
- 通常我们把第一次实验性开发出的软件称为原型（**prototype**）。
- 这种开发模型可以减少由于需求不明给开发工作带来的风险，有较好的效果。
- 相对瀑布模型来说，原型模型更符合人类认识真理的过程和思维，是目前较流行的一种实用的软件开发方法。





① 原型模型的工作机理（反复迭代的过程）





① 原型模型的特点

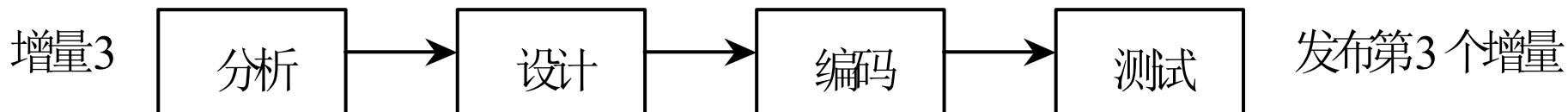
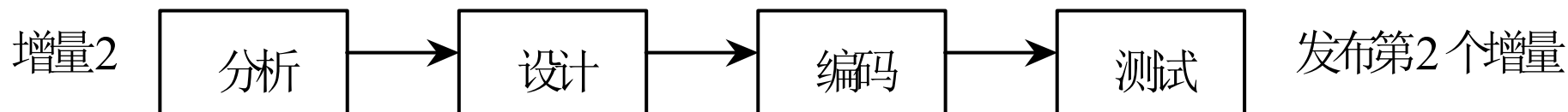
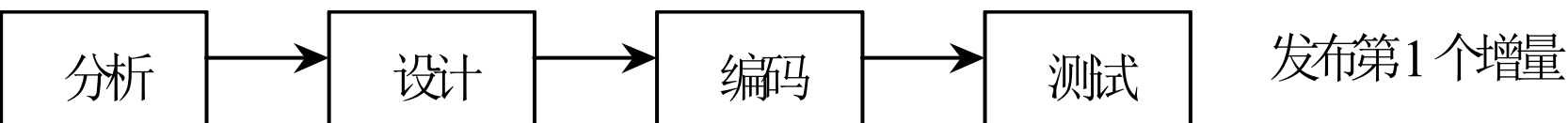
- I. 使开发人员**迅速**建立可运行的软件原型，并在此基础上通过反复的调整和完善的**迭代过程**建立满足用户需求的工作程序。
- II. 第一个原型可能很少真正使用（初始原型很不成熟）
- III. 容易放松对软件的开发管理（因为不确定因素太多，所以随意性较强）；
- IV. 开发者常常因实现上的需要而做出某些折衷处理，以便使原型能尽快工作。





1. 增量模型

- 增量模型融合了瀑布模型的按计划分阶段进行的特征和原型模型的迭代特征





① 增量模型的特点

- I. 各线性序列交错进行，且每个线性序列产生最终产生软件的一个可发布的“增量”
- II. 第一个增量往往是软件的核心，实现了软件的基本功能，后续增量是对软件功能的完善。（如：软件版本的升级、软件补丁）
- III. 采用增量模型可以根据产品的市场需要决定是否开发某个增量（软件功能），因此，可以有计划地控制投资风险和技术风险。

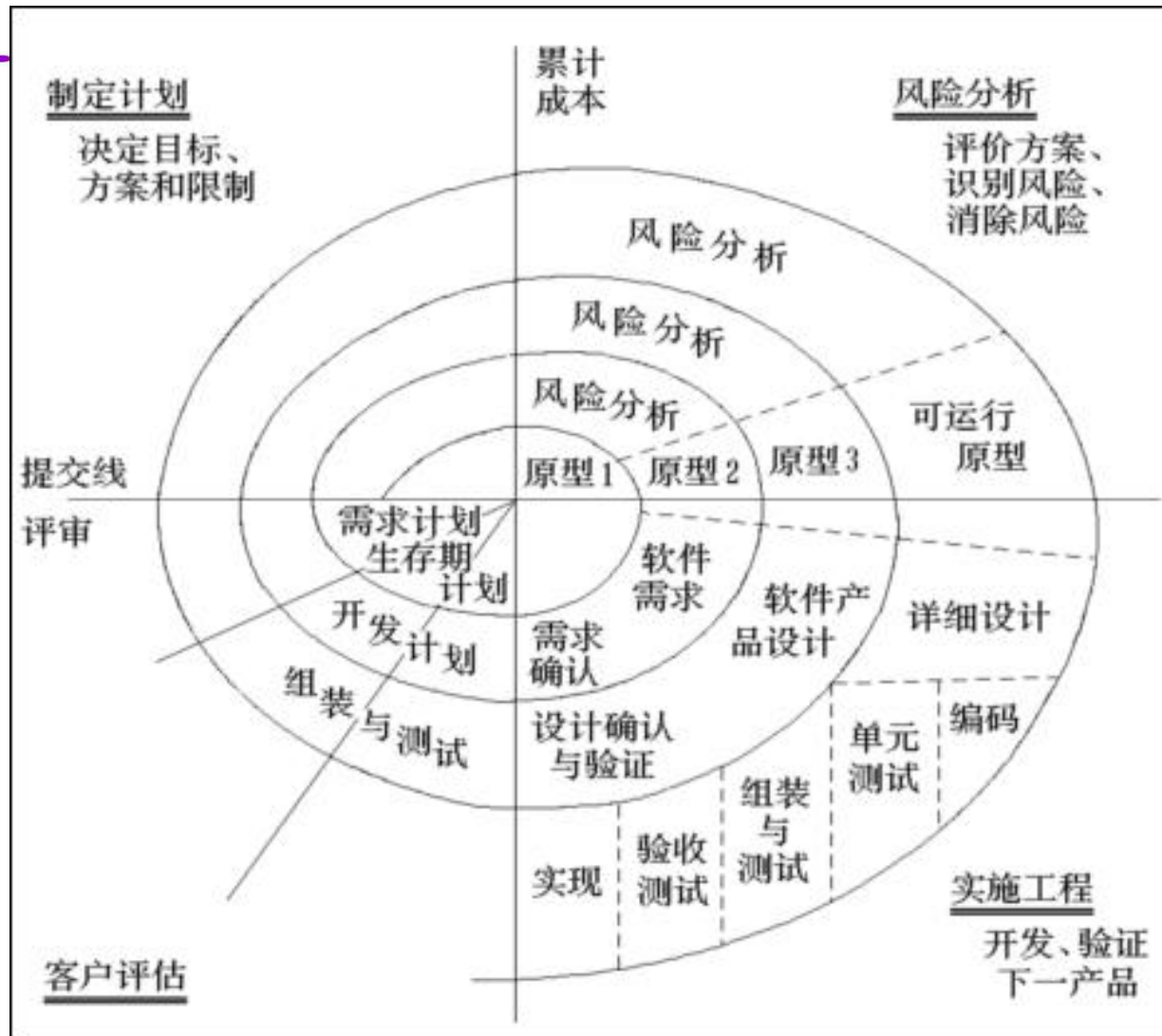




1. 螺旋模型（充分计划好后再实施）

- 由TRW公司的B.Boehm于1988年提出。
- 螺旋模型是瀑布模型与原型模型相结合，并增加两者所忽略的风险分析而产生的一种模型，该模型通常用来指导大型软件项目的开发。
- 它将开发划分为制定计划、风险计划、实施开发和客户评估四类活动。
- 沿着旋线每转一圈，表示开发出一个更完善的新的软件版本。
- 如果开发风险过大，开发机构和客户无法接受，项目有可能就此中止；多数情况下，会沿着螺旋线继续下去，自内向外逐步延伸，最终得到满意的软件产品。



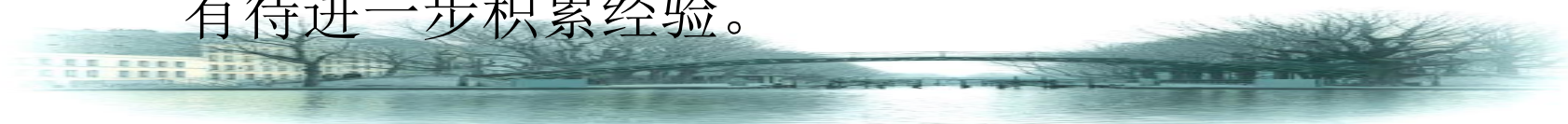


制定计划：确定软件目标，选定实施方案，弄清项目开发的限制条件；
风险分析：分析所选方案，考虑如何识别和消除风险；
实施开发：实施软件开发；
客户评估：评价软件功能和性能，提出修正建议。



- 螺旋模型的特点

- I. 螺旋模型是由上面四个部分组成的迭代模型。螺旋模型的每一周期都包括需求定义、风险分析、工程实现和评审四个阶段。
- II. 开发过程每迭代一次，螺旋线就增加一周，软件开发又前进一个层次，系统又生成一个新版本，而软件开发的时间和成本又有了新的投入。最后得到一个客户满意的软件版本。
- III. 对可选方案和约束的强调有利于已有软件的重用，也有助于把软件质量作为软件开发的一个重要目标，减少了过多测试或测试不足所带来的风险。但是要求许多客户接受和相信并不容易，使用该模型需要具有相当丰富的风险评估经验和专门知识，如果项目风险较大，又未必能及时发现，势必造成重点损失。
- IV. 目前国内许多软件公司还未能及时掌握和运用这种模型，有待进一步积累经验。





3.4 CAD应用软件开发

3.4.1 软件工程的开发流程

3.4.2 软件的文档编制规范





3.4.1 软件工程的开发流程

- 按照国家颁布的计算机软件开发规范（**GB/T8566-1995**），软件的开发应按以下步骤进行：
 1. 可行性研究与项目开发计划
 2. 软件需求分析
 3. 软件设计
 4. 代码实现
 5. 软件测试
 6. 运行与维护





1、可行性计划

确定软件开发目标和总体要求，进行可行性分析，制订可行的开发计划。

这一阶段的任务是**首先明确“要做什么”**，明确软件的功能和目标以及大致规模；**其次研究“是否能做”**，探索要开发软件的难度、深度和广度，估算系统成本和效益，分析开展该项工作的可行性，包括技术、设备、人员以及市场可行性等方面内容。可行性研究的结果是决策者承接或中止该开发项目的重要依据。若研究结果项目可行，则**还要制订开发计划**。





1、可行性计划

2、需求分析

进行系统分析，确定软件功能需求和设计约束。

这一阶段的任务是弄清“必须做什么”。软件开发人员和用户密切配合，充分交流信息，真正准确了解用户的具体要求，得出经过用户确认的系统逻辑模型，避免盲目急于着手设计的倾向。





1、可行性计划

2、需求分析

3、软件设计

确定设计方案，包括软件结构、模块划分、功能分配以及处理流程。

设计阶段分概要设计和详细设计两个步骤。

概要设计的任务是解决“如何做”，考虑多种可能的解决方案，并依据某种令人信服的标准或原则推荐及确定设计方案；然后，进行模块划分，也就是将软件系统按功能划分成许多规模适中的程序集，再将其按合理的层次结构组织起来。

详细设计的任务是解决“如何具体做”，把概要设计的抽象概括解决方案细化、具体化。详细设计还不是编写程序，而是设计出程序的详细规格说明、处理流程。



1、可行性计划

2、需求分析

3、软件设计

4、代码实现

5、软件测试

完成源程序的编码、编译和无语法错误的程序清单。完成程序单元测试。

这个阶段的任务是**编制**出正确的、可读性好的**程序**。开发人员选取适当的程序设计语言，把详细设计的结果翻译成可处理执行的程序，并认真调试、**检测**每一个**程序段**。

实现系统总装测试和确认测试；检查审阅文档；成果评价。





1、可行性计划

2、需求分析

3、软件设计

4、代码实现

5、软件测试

6、运行与维护

软件在运行使用中不断地被维护，根据新提出的需要和运行中发现的问题进行必要且可能的扩充和修改。

通常有四类维护活动：

1) **改正性**维护：诊断和改正运行中发现的软件错误。

2) **适应性**维护：修改软件以适应环境的变化。

3) **完善性**维护：根据用户的要求改进或扩充软件使它更完善。

4) **预防性**维护：修改软件为将来的维护活动做预备。

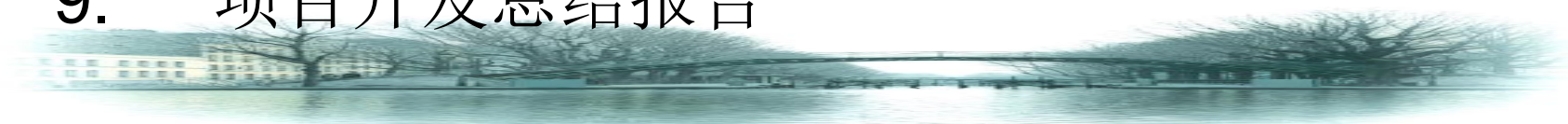
每一项维护活动结束，软件都有不同程度的改进，对于商品化软件来说，都会推出新的版本。



3.4.2 软件的文档编制规范

■ 软件文档通常包括以下9个方面

1. 可行性研究报告
2. 项目开发计划
3. 软件需求说明书
4. 数据要求说明书
5. 概要设计说明书
6. 详细设计说明书
7. 测试计划
8. 测试分析报告
9. 项目开发总结报告





3.5 模块化设计技术

1、模块化原理

- **模块**：是指程序的一个相对独立的部分。

模块化：就是把程序划分为若干个模块，每个模块完成一个子功能，这些模块组织起来形成一个整体，可以完成指定的功能而成为一个软件。

模块化原理：设有两个问题p1、p2和p3，如果p3是p1、p2的和，则对于问题的难度E有：

$$E(p3) > E(p1) + E(p2) \quad (1)$$

从这个不等式可以导出一条结论：

把复杂的问题分解成许多容易解决的小问题，原来的问题也就容易解决了。这就是**模块化的基本原理**。

模块化虽然可以降低模块实现的难度，但模块也不能无限的分割下去，模块划分过细则会造成接口困难和信息管理困难等问题。模块划分一般应注意下列几个方面的问题。



3.5 模块化设计技术

1、模块化原理

2、模块独立性

- **模块独立性：**

模块应该有较高的内聚性，使模块的功能单一，模块中的元素相关性好；同时要使模块的入口唯一，不和其它模块有控制信息耦合，并尽量只和其它模块交换数据信息而不是公共变量。





3.5 模块化设计技术

1、模块化原理

2、模块独立性

3、信息隐蔽化

- **信息隐蔽化：**隐蔽化是指一个模块内的信息对于其他模块来说是不可见的。

原理与作用：预防了模块间的相互干扰，一个模块的修改不会牵涉到其它模块的工作，一个模块的错误也不会传入其他模块。

实现方法：模块内部数据的私有化，除为了软件功能实现所必须的交换信息外，其它一切信息（数据）对于其它模块都是不可访问的。





3.5 模块化设计技术

1、模块化原理

2、模块独立性

3、信息隐蔽化

4、深度、扇出、扇入应适当

● **深度**：是指软件结构中控制的层数。它粗略地反映了一个软件的大小和复杂程度。深度和程序长度之间应该有粗略的对应关系，如果系统不是很大而层数很多则应检查是否有些模块过于简单，能否适当合并。

扇出：是一个模块直接控制的下级模块数目。扇出过大意味着模块过分复杂，控制、协调困难，过小也不好，一般应在3-4最多也不宜超过5-9。

扇入：是直接调用模块的上级模块的数量。提倡高扇入，但不能违背模块独立的原理。



3.5 模块化设计技术

1、模块化原理

2、模块独立性

3、信息隐蔽化

4、深度、扇出、扇入应适当

5、其它模块设计参考规则

1) 模块功能应该是可以预测的。

一个模块只要输入的数据相同，必然产生相同的输出，则该模块的功能是可以预测的。如果模块内带有标志存储器，输出除决定于输入外还和标志有关则不利于模块的测试和维护。

2) 模块规模应适中。

模块规模大不利于理解 and 设计，过小使接口复杂附加开销大，最好能在一页纸内，通常不超过60行语句。

3) 力争降低模块接口的复杂程度。

接口应简单、明确，调用方便，和功能一致。如二次方程求解模块QUAD_ROOT(TBL, X)，以TBL数组输入方程系数，X输出方程根，就不如QUAD_ROOT(A, B, C, ROOT1, ROOT2)简单明确。

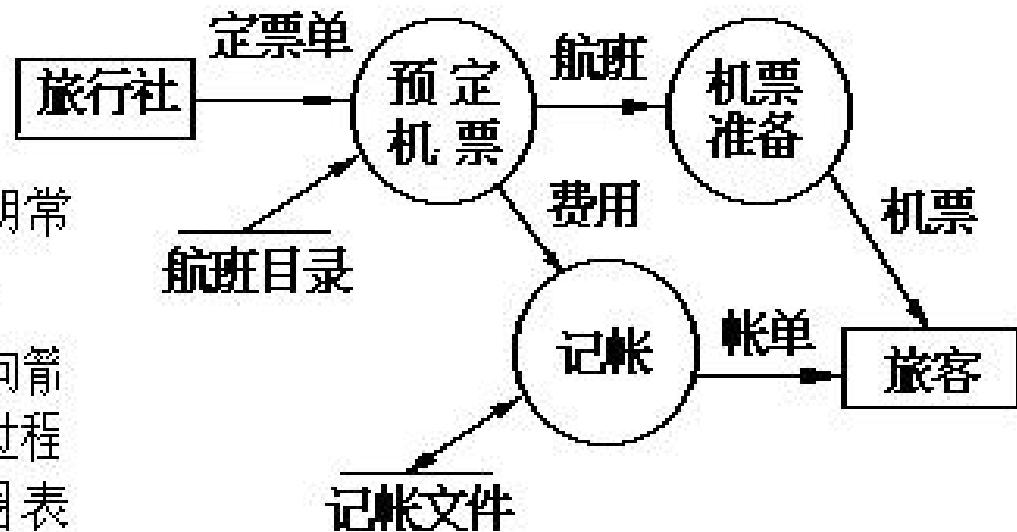


3.6 图形表示技术

1、数据流图

数据流图

- 用途：软件定义时期常用的图形表示方法。
- 方法：数据流用有向箭头表示；数据处理过程称为加工，用圆圈表示；源点和终点是系统之外的实体用方框表示。



机票预定系统的数据流图



3.6 图形表示技术

1、数据流图

2、数据字典



●用途：对数据流图的进一步解释。

●方法：数据词典中主要有三大类条目：数据流条目、文件条目和加工条目。

数据流条目：定义数据流的结构。

定票单=姓名+住址+电话+航班号+日期+始点+终点

航班=日期+航班号+姓名

机票=姓名+日期+航班号+始点+终点+费用

.....

文件条目：定义文件的结构。

航班目录文件={航班号+始点+终点+时间}

组织：按航班号次序排列

记帐文件={旅客+航班+票价+手续费}

组织：按旅客名字的拼音字母顺序排列

加工条目：对数据处理加工方法的进一步说明，一般可用自然语言描述。



3.6 图形表示技术

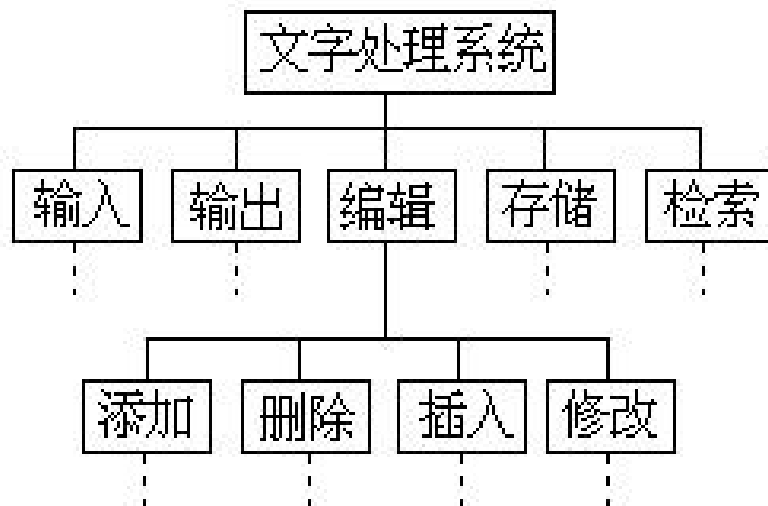
1、数据流图

2、数据字典

3、层次图

层次图

- **用途：**层次图用于描绘软件的层次结构。
- **方法：**一个**方框**代表一个模块；方框间的**连线**表示调用关系。最顶层的模块是系统的主控模块，每个模块调用其下层模块完成其功能。



文字编辑系统的层次结构示意图



3.6 图形表示技术

1、数据流图

2、数据字典

3、层次图

4、I P O表

IPO 表

- **用途：**描述软件模块的输入、处理、输出。
- **方法：**结构如图所示。它详细地记录了模块的输入、输出、调用哪些模块、被哪些模块调用、模块的处理（工作）原理等信息。对编写程序具有重要的指导作用。

IPO表

系统：____ 作者：____
模块：____ 日期：____
编号：____

被调用：

调用：

输入：

输出：

处理：

局部数据元素：

注释：



3.6 图形表示技术

1、数据流图

2、数据字典

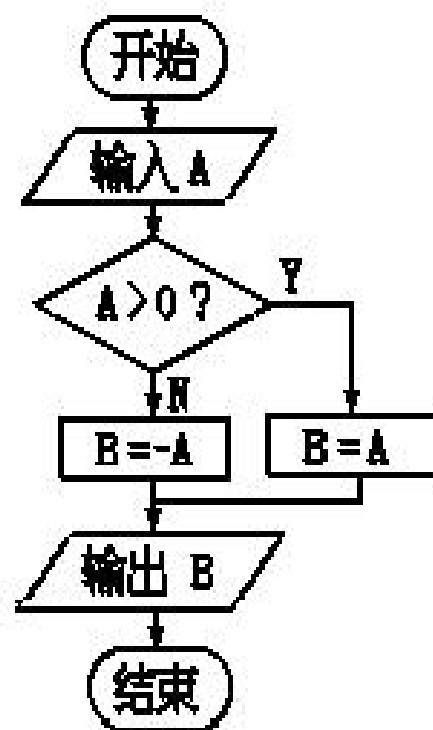
3、层次图

4、I P0表

5、流程图

程序流程图

- **用途：**是程序详细设计阶段常用的辅助工具。又称程序框图。它是历史最悠久、使用最广泛的描述软件设计的方法。
- **方法：**符号使用较乱。但大致以圆角矩形表示开始和结束，矩形表示赋值运算，菱形表示判断。箭头表示数据流方向。





3.6 图形表示技术

1、数据流图

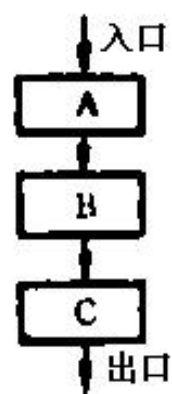
2、数据字典

3、层次图

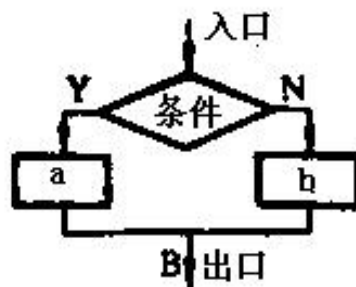
4、I P0表

5、流程图

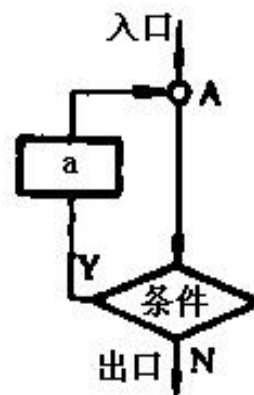
6、程序结构



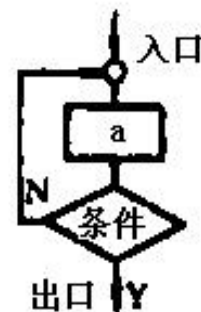
a) 顺序型



b) 选择型



c) 当型循环



d) 直到型循环

图 9-5 结构化程序的基本结构



3.6 图形表示技术

1、数据流图

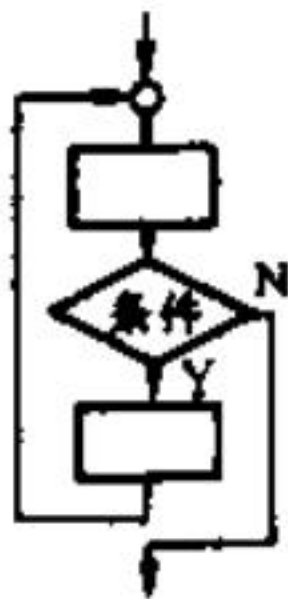
2、数据字典

3、层次图

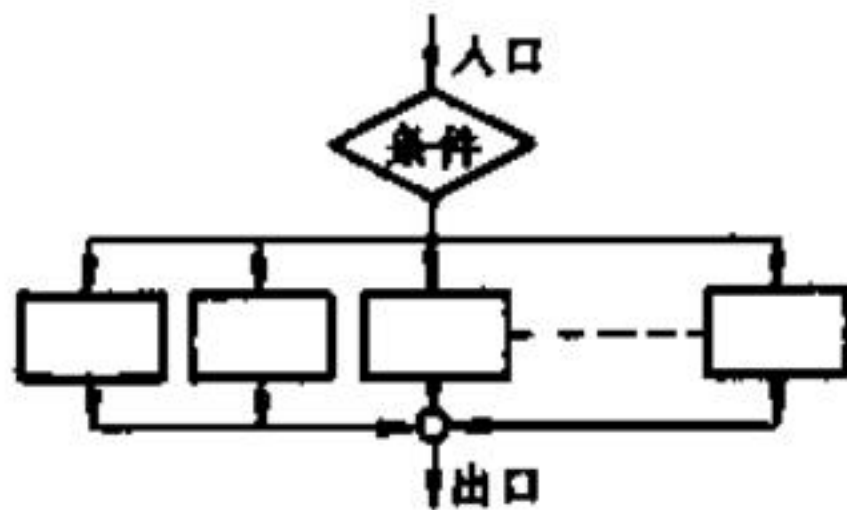
4、I P0表

5、流程图

6、程序结构



a) 复合循环结构



b) 多选择结构

图 9-6 嵌套式结构